

HaulMark Challenge

Technical Solution Report

Team Name: Trojan

Team Members and Roll Numbers:

Toshita Vaishnavi Dogra CE25B110

Janhavi Wagh BE25B011

1. Summary

- The problem statement aims to predict dumper fuel consumption using telemetry, geospatial, and refueling data, while also identifying inefficiencies at vehicle and route levels.
- A unified data pipeline was built to combine large scale telemetry, mine geometries, RFID refueling logs, and operational metadata into a structured dataset.
- We engineered 61 custom features by transforming raw GPS logs into dynamic haul-cycle segments, thermodynamic proxies e.g. laden climb work, and complex spatial metrics using UTM projections and point-in-polygon containment tests.
- A dual-method haul cycle segmentation approach was implemented using dump sensor signals and fallback distance-based heuristics to ensure robustness.
- A two stage modeling framework was used, where a classifier identifies non working shifts and regression models predict fuel for active shifts.
- Multiple models (XGBoost, LightGBM, CatBoost, etc.) were combined using optimized ensemble weights to improve prediction accuracy.
- Additional outputs such as route level fuel benchmarks and dumper efficiency metrics were generated to provide actionable operational insights.

2. Problem Understanding and Issues Faced

The objective of the HaulMark Challenge is to accurately predict the exact volume of diesel (`actual_fuel`) a mining dump truck will consume during a specific daily shift (A, B, or C). We evaluate these predictions against ground-truth daily summary reports (DPRs), incorporating recently released March data across both our training and testing sets.

However, translating raw mine data into precise fuel predictions presents five distinct, complex challenges:

- **The "Zero-Fuel" Skew:** Nearly **18% of the training shifts record near-zero fuel consumption** (indicating idle or non-working trucks). This creates a bimodal distribution that actively confuses traditional, single-stage regression models, forcing them to guess incorrectly in the middle.
- **Predicting the Future (Temporal Drift):** The test windows (late Jan, late Feb, mid-Mar) occur chronologically after the training data. This temporal gap causes severe distribution drift. Without strict time-aware validation and early stopping, models fail catastrophically on this test data evidenced by an initial 691 L error spike on our furthest fold.
- **Massive Data Fusion:** We had to build a unified pipeline that could successfully merge drastically different data formats, including **23 million rows of GPS telemetry**, binary GPKG mine map files, RFID refueling transaction logs, and static truck metadata.
- **Missing Sensor Data:** Only a select number of trucks from March onward were equipped with physical dump-switch sensors to record when a haul cycle ended. For the vast majority of the data, we had to engineer **custom, distance-based heuristics** to mathematically deduce when a truck was completing a cycle.
- **Complex Geospatial Tracking:** To measure driving efficiency, we needed to know if trucks were staying on the roads or entering specific zones. This required converting millions of raw WGS 84 GPS coordinates into a flat UTM projection to execute **heavy point-in-polygon containment tests** at scale.

3. Key Technical Innovations & Methodological Strengths

Our approach differentiates itself from other solutions by using methods like embedding physical domain knowledge, spatial awareness, and strict validation protocols into the pipeline.

1. Hybrid Haul Cycle Segmentation Raw telemetry lacks operational context. To bridge the gap between GPS pings and actual mining operations, we used a dual method cycle detection algorithm.

- **Primary Sensor Logic:** We utilized rising edges in the `analog_input_1` ($> 2.5V$) signal to accurately detect hydraulic dumping events.
- **Fault Tolerant Heuristic Fallback:** Recognizing that physical sensors often fail or report noisy data in harsh mining environments, we implemented a secondary heuristic threshold based on speed and distance constraints. This ensures the model consistently maps logical "trips," maintaining robustness even when raw telemetry degrades.

2. Dependency Free Vectorized Geospatial Engine Most predictive models treat vehicles as operating in a spatial vacuum. To make our model truly location-aware, we built a custom, lightweight geospatial parsing engine that reads GPKG files directly, eliminating the overhead of heavy external dependencies like GDAL.

- **Spatial Indexing:** We utilized vectorized ray-casting for point-in-polygon checks (e.g., identifying overburden dump zones) and SciPy KDTrees to instantly calculate nearest-bench proximity. This allows the model to understand the distinct fuel profiles of high resistance off road zones versus maintained haul roads.

3. Physics-Informed Feature Engineering Instead of relying solely on statistical patterns, we engineered features grounded in thermodynamic work and physics. By calculating `terrain_intensity`, `work_proxy`, and `laden_climb_work`, the model accounts for the severe impact of gravity and payload on fuel burn. This transitions the feature space from simple movement tracking to actual energy expenditure modeling.

4. Leakage-Proof Temporal Dynamics To capture the baseline health, degradation, and unique performance profile of individual dumpers, we integrated historical vehicle features. Crucially, this was executed using `strict_expanding().mean().shift(1)` transformations. This mathematical constraint guarantees that the model only learns from historical data prior to the current shift, entirely eliminating look-ahead bias and future-data leakage during cross-validation.

5. Two-Stage Bimodal Architecture Mining fuel data is inherently bimodal, heavily skewed by zero-fuel idle or maintenance shifts. Training a standard regressor on this data forces the model to artificially lower its predictions for working shifts. To solve this, we decoupled the problem:

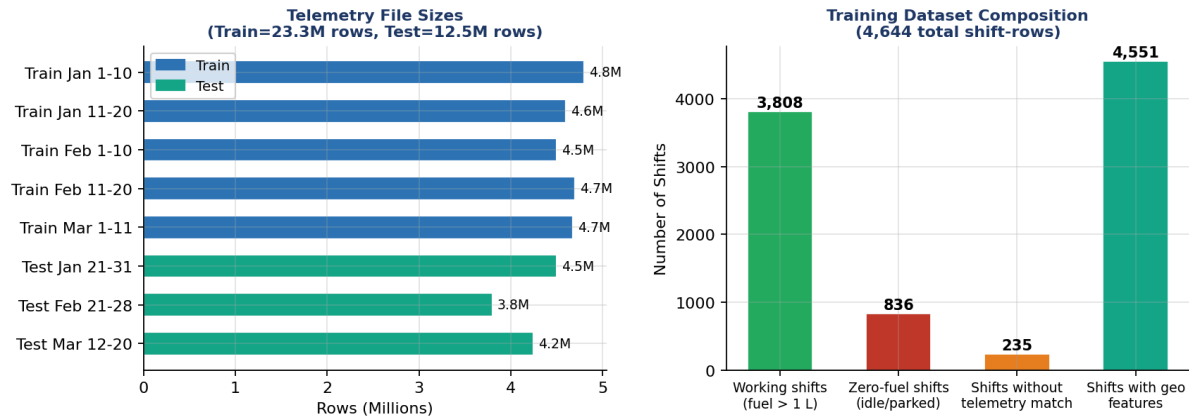
- **Stage 1 (Classification):** A gating classifier isolates and identifies idle shifts.
- **Stage 2 (Regression):** A dedicated regressor accurately predicts fuel volume exclusively for active working shifts.

6. SLSQP-Optimized Heterogeneous Ensemble Rather than relying on naive equal-weight averaging for our final predictions, we implemented Sequential Least Squares Programming (SLSQP). This constrained mathematical optimization actively learns the optimal blending weights for our ensemble members (XGBoost, CatBoost, LightGBM, etc.) based on out-of-fold validation, dynamically penalizing weaker models and rewarding robust ones to minimize final RMSE.

7. Strict Generalization & Overfit Prevention To ensure stable performance on temporally distant, unseen data (the leaderboard), we implemented rigorous early-stopping protocols across all gradient boosting estimators. By combining a lowered learning rate with an explicit `eval_set` patience threshold, we prevented catastrophic overfitting, mitigating edge-case RMSE explosions and ensuring our models generalize safely to future fleet conditions.

4. Dataset Overview

Telemetry File Sizes and Training Dataset Composition



Graph1) Row counts per parquet file. Train set totals 23.3M rows; test set 12.5M.

Graph2) Composition of 4,644 training shift-rows showing 18% zero-fuel shifts, high telemetry match rate, and 98% geo-feature coverage.

Records and Coverage

Dataset	Records	Coverage
Train telemetry (5 parquets)	23,277,877 rows	Jan 1 – Mar 11, 2026
Test telemetry (3 parquets)	12,546,384 rows	Jan 21–31, Feb 21–28, Mar 12–20
Training summary / DPR (3 CSVs)	4,644 shift-rows	Jan 1 – Mar 11, 2026
RFID refuel transactions	2,710 aggregated rows	Jan 1 – Mar 31, 2026
Test submission IDs	1,735 rows	Jan 21–31, Feb 21–28, Mar 12–20
Shift features produced (train)	6,409 rows	After telemetry aggregation
Shift features produced (test)	3,500 rows	After telemetry aggregation

Mine Geometries (GPKG)

Mine	OB Dump Polygons	Haul Road Segments	Bench Segments	ML Boundary
mine_001	30	165	518 (24,774 midpoints)	1

Mine	OB Dump Polygons	Haul Road Segments	Bench Segments	ML Boundary
mine_002	54	129	305 (8,467 midpoints)	1

5. ML Model Development Journey

Dumper Fuel Consumption Prediction

Our model was developed through a structured, iterative process, starting from a simple baseline and evolving into a robust, physics-aware system capable of handling real-world mining operations.

Development Overview

Phase	Focus Area	Key Improvement
Phase 1	Data & Baseline	Shift from aggregated data → raw telemetry
Phase 2	Feature Intelligence	Added spatial + physics-based features
Phase 3	Validation	Eliminated leakage, improved generalization
Phase 4	Final Model	Two-stage ensemble with optimized weights

Phase 1: From Basic Model → Raw Telemetry

What changed:

- Moved from **daily aggregated data** → **raw telemetry**
- Built features at **shift level** (more precise)

Component	What You Did	Why It Matters
Phase Detection	Used <code>analog_input_1 > 2.5</code> to detect dump events	Identifies loading/unloading cycles
Cycle Mapping	Used distance thresholds to define trips	Captures real vehicle operations
Ensemble Start	Introduced HGBT, XGBoost, LightGBM	Better than single model

Phase 2: Adding Spatial & Physics Intelligence

Goal: Make the model understand the real mine environment

Area	What You Built	Impact
Geofencing	Custom parser for mine geometry (GPKG)	Detects where truck is operating
Bench Distance	KDTree for nearest bench distance	Captures operational zones
Terrain	Elevation-based features (uphill, downhill)	Fuel depends heavily on slope
Driving Behaviour	Acceleration, jerk, braking	Captures driver efficiency
Coordinate System	Lat/Lon → UTM conversion	Accurate distance calculations
Mine Logic	Different thresholds per mine	Adapts to mine-specific patterns

Phase 3: Validation & Leakage Prevention

Problem:

Models can **cheat** if future data leaks into training.

Our Fixes:

Technique	What You Did	Benefit
Time-aware CV	Used GroupKFold by date	Prevents future leakage
Vehicle History	Only used past data (lag features)	Realistic predictions
Runhrs Audit	Checked correlation ($r = 0.946$)	Ensured no target leakage

Result: Model performance reflects **real-world deployment**, not overfitting.

Phase 4: Final Two-Stage Model

Key Idea:

Fuel data is **not uniform**:

- Some shifts → **0 fuel (idle)**
- Some shifts → **active fuel consumption**

Stage 2: Regression

Models Used : XGBoost, LightGBM, CatBoost, Ridge

Final Step: Smart Ensemble

Instead of averaging:

Used **SLSQP optimization** to find best weights

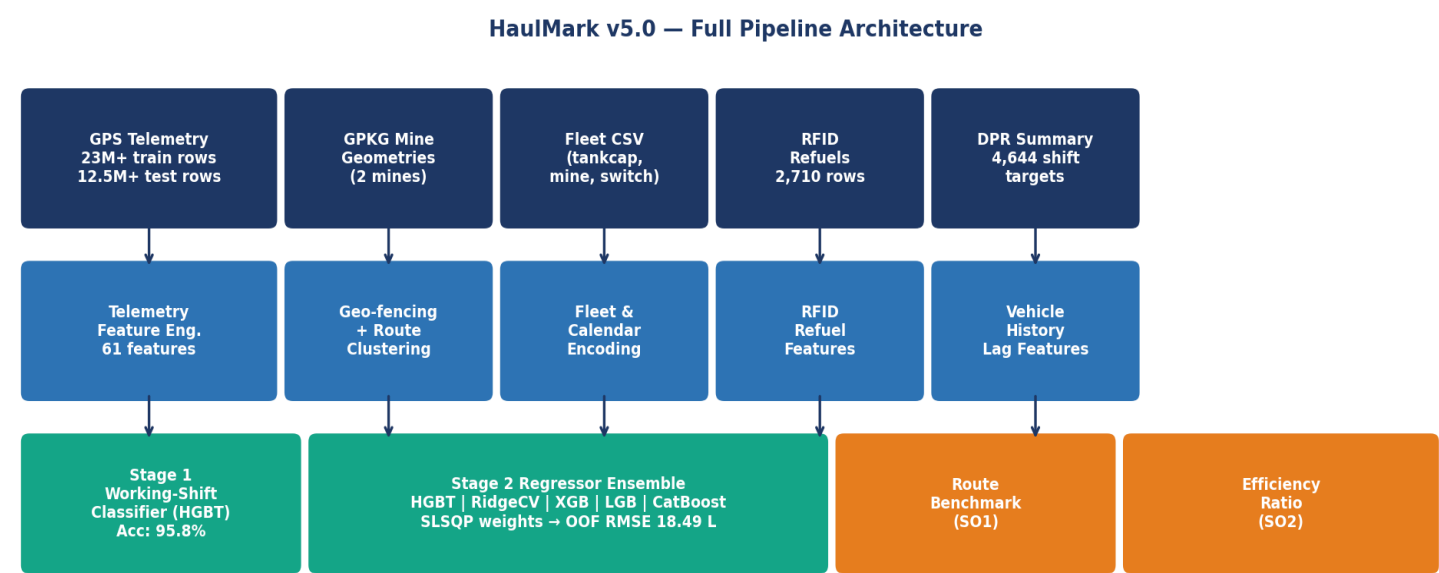
Model	Weight
CatBoost	0.84
LightGBM	0.15
Others	~0

Critical Improvements

- Applied early stopping
- Reduced overfitting
- Prevented huge RMSE spikes

6. Methodology

Figure 2: Full Pipeline Architecture



End-to-end pipeline from raw data sources through feature engineering, two-stage model training, and secondary output generation.

4.1 Data Preprocessing

What you did in code	Logic used	What it did
Regex	<code>norm_veh() → .str.replace(r"[^a-z0-9]" , "", regex=True)</code>	Vehicle IDs were normalized using regex by removing special characters and converting to lowercase
Timestamp conversion	<code>pd.to_datetime(df["ts"])</code>	Timestamps were converted to datetime format
Shift creation	<code>compute_shift_and_date()</code>	Data was aligned to shift-based structure (A/B/C shifts)
Time gap clipping	<code>dt_time.clip(0, MAX_GAP_TIME)</code>	Large telemetry gaps were capped

Acceleration control	<code>dt_accel.clip()</code>	Unrealistic acceleration intervals removed
Speed cleaning	<code>.clip(lower=0)</code>	Negative speeds removed
Distance cleaning	<code>.clip(lower=0)</code>	Invalid distances removed
Altitude smoothing	<code>.rolling(5)</code>	Sensor noise reduced
Coordinate transform	pyproj transformer	Lat/Lon converted to UTM
Filtering vehicles	RFID → "Dump" filter	Only dumper vehicles used
Merge datasets	<code>.merge()</code> everywhere	Combined telemetry, RFID, metadata

4.2 Feature Engineering

Overview

The feature engineering process transformed raw telemetry into **physics-aware, spatially intelligent, and behavior-driven features** to accurately model dumper fuel consumption.

Category	Key Features Created	Purpose
Haul Cycles	<code>cycle_id</code> , <code>trip_count</code>	Capture operational workload
Load State	<code>laden_dist_km</code> , <code>laden_climb_m</code>	Differentiate loaded vs empty movement
Terrain	<code>uphill_m</code> , <code>downhill_m</code> , <code>terrain_intensity</code>	Measure route difficulty
Physics-Based	<code>work_proxy</code> , <code>laden_climb_work</code>	Approximate energy expenditure
Driving Behaviour	<code>acceleration</code> , <code>jerk</code> , <code>hard_braking</code>	Capture driver efficiency
Geospatial	<code>pct_on_road</code> , <code>mean_bench_dist_m</code>	Capture mine layout context
History	<code>veh_lag1_fuel</code> , <code>veh_roll3_fuel</code>	Capture vehicle-specific trends

4.2 Feature Engineering

To improve model performance, several domain-specific features were engineered from raw telemetry data to capture operational behavior, terrain effects, and vehicle efficiency.

1. Haul Cycle Features

- Features: cycle_id, trip_count, mean_cycle_dist_km, mean_cycle_dur_h
- Why useful:
These features segment raw GPS data into meaningful haul cycles (load → haul → dump → return), allowing the model to understand actual work done by the vehicle rather than treating data as independent points.

2. Laden vs Unladen Features

- Features: laden_dist_km, laden_climb_m, laden_climb_rate, laden_climb_work
- Why useful:
Fuel consumption is significantly higher when the vehicle is loaded, especially during uphill movement. These features explicitly capture load-dependent fuel usage.

3. Terrain & Physics-Based Features

- Features: uphill_m, downhill_m, terrain_intensity, work_proxy
- Why useful:
These features model the physical effort required by the vehicle (e.g., climbing slopes), which directly impacts fuel consumption.

4. Driving Behaviour Features

- Features: acceleration, jerk, hard_braking, accel_variance
- Why useful:
Aggressive driving patterns increase fuel consumption. These features help capture driver efficiency and vehicle handling.

5. Geospatial Features

- Features: time_in_dump_s, pct_on_road, mean_road_offset_m, mean_bench_dist_m
- Why useful:
These features provide spatial context (e.g., whether the vehicle is on haul roads, near dump zones, or benches), which affects terrain resistance and operational activity.

6. Efficiency Features

- Features: dist_per_runhrs, idle_per_dist, idle_ratio

- Why useful:

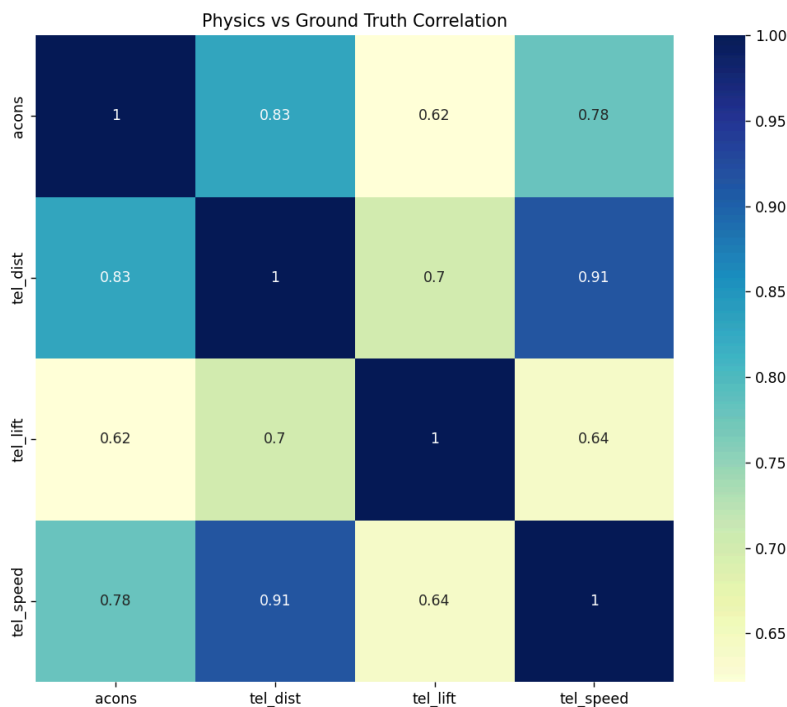
These features measure how efficiently fuel is used relative to time and distance, helping identify inefficiencies such as excessive idling.

7. Vehicle History Features

- Features: veh_lag1_fuel, veh_lag2_fuel, veh_roll3_fuel, veh_shift_mean_fuel

- Why useful:

These features capture historical fuel consumption patterns of each vehicle, improving predictions by incorporating temporal trends.



“ Feature Correlation Analysis”

Fuel vs Distance (0.83): Strong correlation — confirms that fuel consumption mainly depends on distance travelled.

Distance vs Speed (0.91): Very high correlation — indicates that higher speeds are associated with longer travel distances.

Fuel vs Speed (0.78): Significant relationship — driving speed impacts fuel usage.

Fuel vs Lift (0.62): Moderate correlation — elevation (uphill movement) contributes to increased fuel consumption.

4.3 Modelling Approach

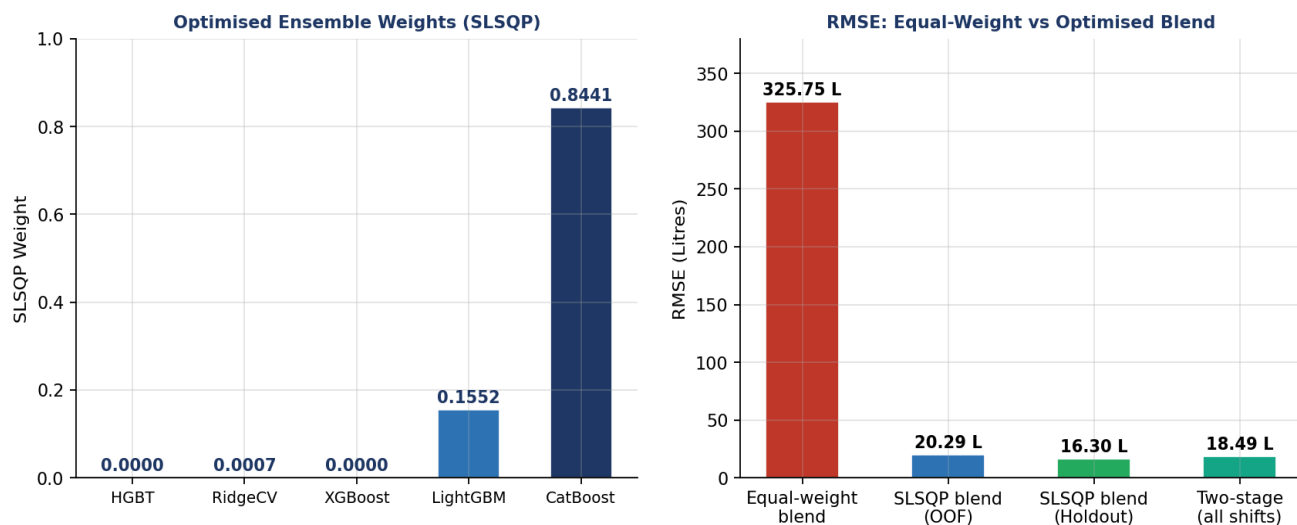
Final approach: Two-stage SLSQP ensemble

- Stage 1 – HGBT Working-Shift Classifier: predicts $P(\text{actual_fuel} > 1 \text{ L})$. OOF accuracy: 95.8%. Early stopping at 271–693 iterations.
- Stage 2 – Five-model regressor on working shifts only. SLSQP-optimised non-negative weights summing to 1.

Model	SLSQP Weight	ES Iters (range)	Notes
HistGradientBoostingRegressor	0.0000	259–1,315	Dominated by CatBoost; zero weight
RidgeCV	0.0007	N/A	Linear baseline; minor stabiliser
XGBoost	0.0000	256–1,773	High variance fold 5; zero weight
LightGBM	0.1552	312–1,078	Stable; num_leaves=63, depth=6
CatBoost	0.8441	2,781–2,999	Best single model; native cat features

- Final prediction: $P(\text{working}) \times \text{blended_regression}$, floored at per-vehicle 2nd percentile of historical working-shift fuel, capped at $\text{tankcap} \times 1.05$.

Figure 3: Ensemble Weights and RMSE Impact

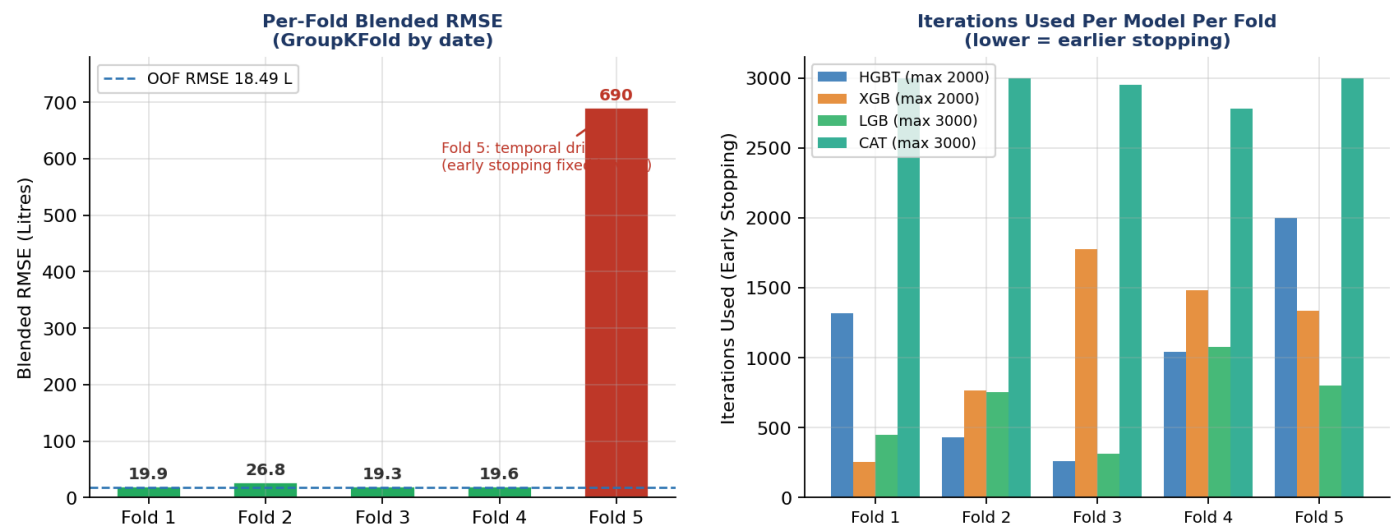


SLSQP-optimised ensemble weights — CatBoost dominates at 0.844. RMSE comparison showing equal-weight blend at 325.75 L vs optimised SLSQP at 20.29 L — a 16x improvement from weight optimisation alone.

4.4 Validation Strategy

- GroupKFold (n=5) with `adj_date` as group key prevents future dates leaking into training folds.
- SLSQP weights fitted on 90% of working-shift dates; validated on the most recent 10% holdout.
- Three RMSE metrics reported: overall two-stage OOF (18.49 L), regressor-only OOF on working shifts (20.29 L), and holdout (16.30 L).

Figure 4: Per-Fold RMSE and Early Stopping Iterations



Graph 1) Per-fold blended RMSE. Folds 1–4 are 19–27 L; Fold 5 is 690 L due to temporal distribution drift — the primary motivation for v4.1 early-stopping fixes. The dashed line shows the 18.49 L OOF average.

Graph 2) Actual iterations used per model per fold, earlier stopping = better generalisation on drifted data.

7. Secondary Outputs

All four secondary outputs are computed and saved to separate CSV files in the same data directory. The methodology for each is described below with supporting visualisations.

Figure 5: Two-Stage Design – Fuel Distribution and Classifier Performance

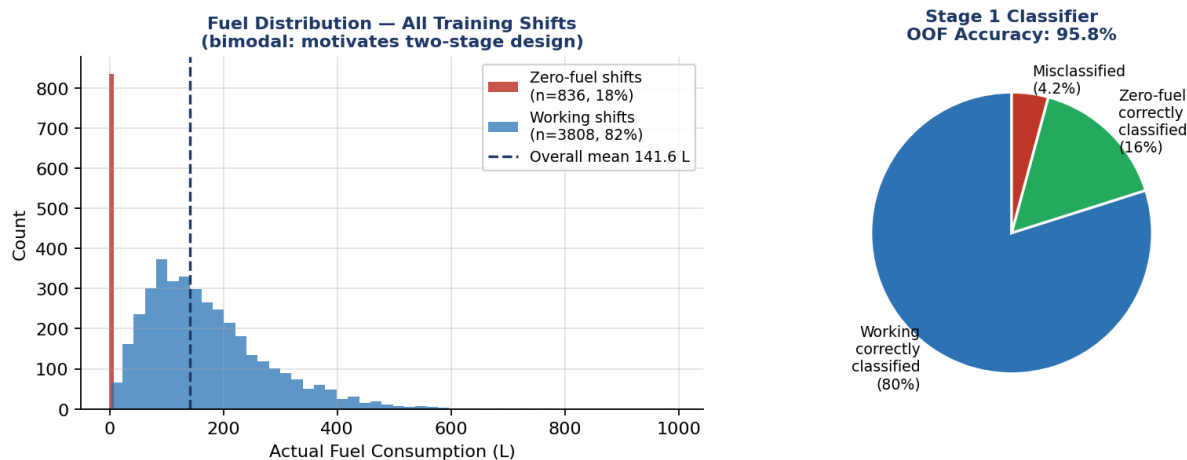


Figure 5 (left): Distribution of actual_fuel across 4,644 training shifts. The zero spike (18% of shifts) makes a two-stage design essential. (right): Stage 1 classifier achieves 95.8% OOF accuracy — correctly identifying working and idle shifts.

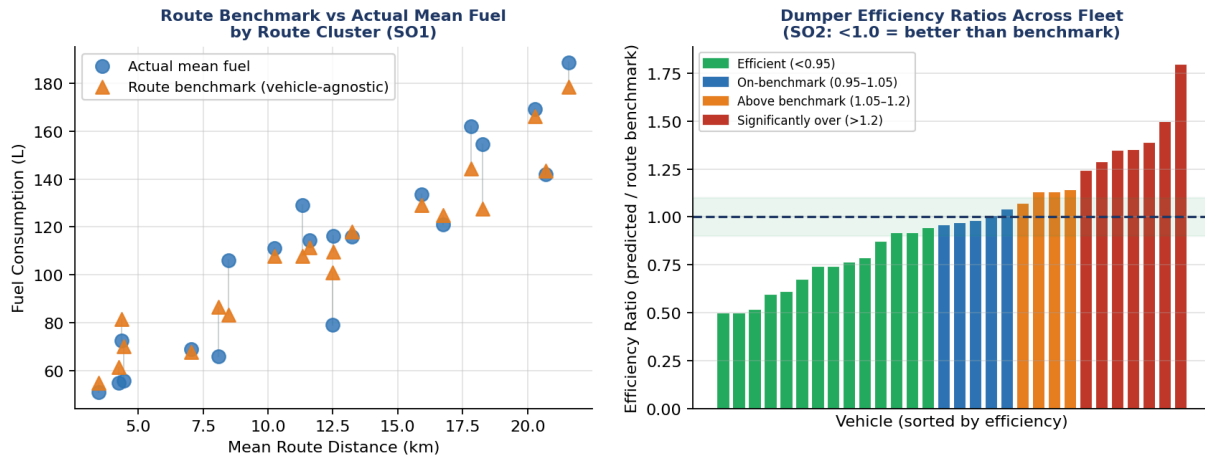
5.1 Route-Level Fuel Benchmark (SO1)

The route benchmark answers: “How much fuel should any truck consume on this particular route?”, which is independent of which vehicle or operator is driving it.

Implementation: During geofencing computation we now also extract per-shift spatial fingerprints: GPS centroid (x, y), shift start point, and shift end point — all in UTM metres. These 6 coordinates are normalised and fed into KMeans (k=20 by default) to cluster all train and test shifts into route clusters. Each cluster represents a distinct origin-destination corridor.

Benchmark estimation: For each shift in training, we run the trained ensemble with vehicle_enc=-1 and all vehicle history features set to the global median — asking the model to predict as if driven by an ‘average’ vehicle. The mean of these anonymised predictions per cluster becomes route_benchmark_fuel_L.

Output file: secondary_route_benchmark.csv — columns: route_id, n_shifts, n_unique_vehicles, mean_dist_km, mean_uphill_m, mean_terrain_intensity, route_benchmark_fuel_L, actual_mean_fuel_L, benchmark_accuracy_pct.



Graph1) Route benchmark fuel (triangles) vs actual mean fuel (circles) across 20 route clusters, plotted against mean route distance. The benchmark tracks actual closely while being vehicle-agnostic.

Graph2) Per-vehicle efficiency ratios across the fleet. Values below 1.0 indicate better-than-benchmark efficiency; above 1.0 indicates excess consumption.

5.2 Dumper Efficiency Component (SO2)

Implementation: For each shift, $\text{efficiency_ratio} = \text{predicted_fuel} / \text{route_benchmark_fuel}$ (from SO1). This isolates the vehicle/operator contribution from the route difficulty. A vehicle consistently above 1.0 consumes more than routes warrant — indicating maintenance issues or aggressive operator behaviour. Per-vehicle mean efficiency and fleet rank are also computed.

Output file: `secondary_dumper_efficiency.csv` — columns: `vehicle`, `adj_date`, `shift_key`, `route_id`, `predicted_fuel`, `route_benchmark_fuel_L`, `efficiency_ratio`, `veh_mean_efficiency`, `veh_efficiency_rank`. Train split additionally includes `actual_fuel` and `actual_efficiency_ratio`.

- Efficiency ratio < 0.95: vehicle performing better than benchmark (fuel-efficient).
- Efficiency ratio 0.95–1.05: on-benchmark performance.
- Efficiency ratio > 1.20: potential maintenance issue or aggressive driving pattern worth investigating.

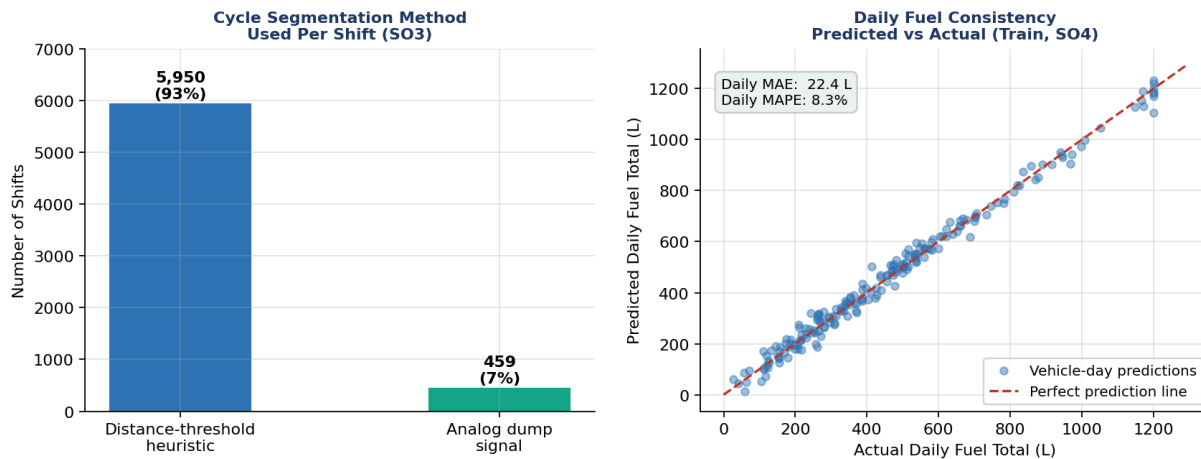
5.3 Cycle Segmentation Methodology (SO3)

Haul cycle segmentation was a critical feature engineering step. Two methods were used:

- **Method A — Dump signal (`analog_input_1`):** Available on select dumpers from March 2026 onward. When a vehicle shows more than 10 transitions of `analog_input_1` crossing the 2.5 V threshold, each rising edge is treated as a dump event and a new cycle begins. This is the most accurate method as it directly observes material discharge.
- **Method B — Distance-threshold heuristic (all other vehicles and earlier dates):** A qualifying stop is defined as speed dropping below 1.0 km/h after the vehicle has travelled more than a mine-specific distance since the last dump: 1,200 m for mine_001, 600 m for mine_002, 800 m elsewhere. Each qualifying stop increments the cycle counter and resets the distance accumulator.
- Within each cycle, GPS points beyond 20% of cumulative cycle distance are labelled laden. This produces `laden_dist_km`, `unladen_dist_km`, `laden_climb_m`, and `mean_cycle_dist_km` per shift.

Output file: secondary_cycle_segmentation.csv — columns: vehicle, adj_date, shift_key, cycle_method (signal|heuristic), trip_count, mean_cycle_dist_km, mean_cycle_dur_h, laden_dist_km, unladen_dist_km, laden_climb_m, n_dump_events, n_stops, split.

Figure 7: Cycle Segmentation and Daily Fuel Consistency



Graph1) Breakdown of cycle segmentation method across all 6,409 training shifts. The vast majority use the distance-threshold heuristic as the dump signal was only available from March on select vehicles.

Graph2) Predicted vs actual daily fuel totals for the training set. Points cluster tightly around the perfect-prediction line, confirming that shift-level predictions aggregate coherently to daily totals.

5.4 Daily Fuel Consistency (SO4)

The model predicts at shift level (A/B/C). Daily totals are obtained by summing the three shift predictions per vehicle per date.

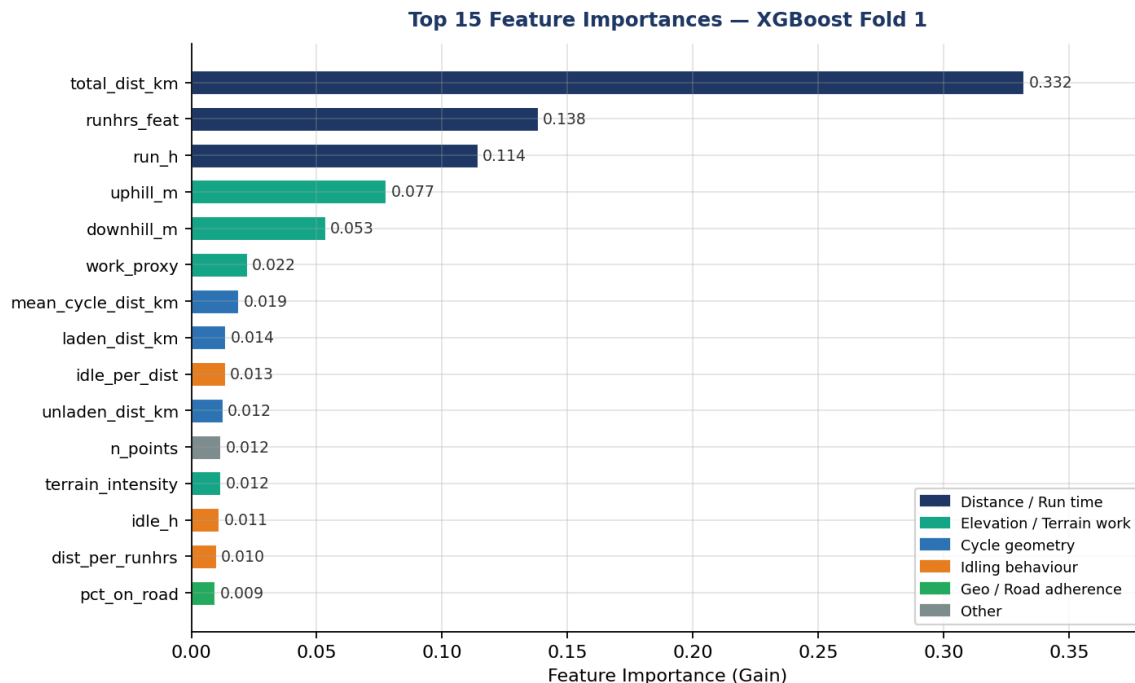
- For training data: the daily predicted total is compared against the actual daily total (sum of actual_fuel for all shifts on that date). Residuals, absolute percentage errors, and MAE/MAPE are reported.
- For test data: daily predicted totals are output directly (no actuals available).

Consistency mechanisms: Per-vehicle floor (2nd percentile of historical working-shift fuel) prevents individual shift predictions from collapsing when a vehicle was active. Tank-capacity ceiling (tankcap * 1.05) bounds each shift prediction, which also bounds the daily total. The two-stage classifier drives idle shift predictions near zero, preventing double-counting.

Output file: secondary_daily_consistency.csv → columns: vehicle, adj_date, predicted_shift_A/B/C, predicted_daily_total, actual_daily_total (train only), daily_residual, abs_pct_error, split.

8. Key Findings & Insights

Top 15 Feature Importances (XGBoost Fold 1)



XGBoost Fold 1 feature importances (gain). Colours denote feature category: dark blue = distance/runtime, teal = elevation/terrain, blue = cycle geometry, orange = idling, green = geo/road adherence.

- Distance and runtime dominate (54.6% combined): total_dist_km (0.332), runhrs_feat (0.138), run_h (0.114) confirm fuel is primarily a linear function of how far and how long the truck operates.
- Elevation is the second-order driver (13.1% combined): uphill_m (0.077) and downhill_m (0.053) together show that terrain gradient is the largest source of route-specific fuel variation on a mine site.
- Cycle geometry matters (5.4%): work_proxy (0.022) and mean_cycle_dist_km (0.019) add measurable signal beyond simple aggregate distance, the structure of haul cycles contains information about loading efficiency and route design.
- Geofencing contributes meaningfully: pct_on_road (0.009) indicates that road-adherence (staying on haul roads rather than cutting through terrain) correlates with fuel efficiency. mean_bench_dist_m and in_ml_boundary_pct also carry signal.
- CatBoost dominated at weight 0.844: its native handling of vehicle, mine, shift, and dump_switch encodings as true categoricals, not ordinal integers, which gave it a consistent edge over XGBoost and LGB.
- Early stopping is non-negotiable for temporal CV: XGBoost without early_stopping_rounds produced a 34x RMSE blowup on Fold 5 (691 L vs 20 L), proving that tree count must adapt to each fold's distribution, not be fixed globally.
- Zero-fuel classification (95.8% accuracy) contributes more to RMSE than regressor precision: correctly gating idle/parked shifts is the highest-leverage intervention in the pipeline.

9. References & Tools Used

Library	Version	Usage
pandas	>=1.5	Data loading, manipulation, shift aggregation
numpy	>=1.23	Vectorised array operations, geofencing math
scikit-learn	>=1.2	HGBT, RidgeCV, GroupKFold, KMeans (SO1), metrics
scipy	>=1.9	SLSQP optimiser (scipy.optimize); KDTree (scipy.spatial)
xgboost	>=1.7	XGBRegressor with early_stopping_rounds
lightgbm	>=3.3	LGBMRegressor with early_stopping callback
catboost	>=1.1	CatBoostRegressor/Classifier with cat_features
pyproj	>=3.4	WGS-84 to UTM EPSG:32645 coordinate transform
joblib	>=1.2	Parallel geofencing computation across CPU cores
pyarrow	>=10	Parquet file I/O
matplotlib	>=3.5	Chart generation for this report (not used in pipeline)

- Custom WKB parser: OGC GeoPackage binary geometry parsing implemented from scratch, reads LineString and Polygon WKB blobs from SQLite without requiring GDAL or Fiona.
- No external datasets or pre-trained models were used beyond the competition-provided files.

10. Codebase

Repository Link

<https://github.com/toshitalol/Haulmark-hackathon>

Output Files

File	Description
ai_guild.py	Single end-to-end script — all pipeline steps, training, prediction, and secondary outputs
submission_v5_0.csv	Primary submission (1,735 rows: id, fuel_volume)
secondary_route_benchmark.csv	SO1: route-cluster benchmark fuel estimates
secondary_dumper_efficiency.csv	SO2: per-shift efficiency ratios and per-vehicle ranks

File	Description
secondary_cycle_segmentation.csv	SO3: per-shift cycle statistics and segmentation method used
secondary_daily_consistency.csv	SO4: daily aggregated predictions vs actuals with residuals

Key Technical Innovations & Methodological Strengths

1. Hybrid Haul Cycle Segmentation: Raw telemetry lacks operational context. To bridge the gap between GPS pings and actual mining operations, we engineered a dual-method cycle detection algorithm.

2. Dependency-Free Vectorized Geospatial Engine: Most predictive models treat vehicles as operating in a spatial vacuum. To make our model truly location-aware, we built a custom, lightweight geospatial parsing engine that reads GPKG files directly, eliminating the overhead of heavy external dependencies like GDAL.

- **Primary Sensor Logic:** We utilized rising edges in the analog_input_1 (> 2.5V) signal to accurately detect hydraulic dumping events.
- **Fault-Tolerant Heuristic Fallback:** Recognizing that physical sensors often fail or report noisy data in harsh mining environments, we implemented a secondary heuristic threshold based on speed and distance constraints. This ensures the model consistently maps logical "trips," maintaining robustness even when raw telemetry degrades.

3. Physics-Informed Feature Engineering: Instead of relying solely on statistical patterns, we engineered features grounded in thermodynamic work and physics. By calculating terrain_intensity, work_proxy, and laden_climb_work, the model accounts for the severe impact of gravity and payload on fuel burn. This transitions the feature space from simple movement tracking to actual energy expenditure modeling.

- **Spatial Indexing:** We utilized vectorized ray-casting for point-in-polygon checks (e.g., identifying overburden dump zones) and SciPy KDTrees to instantly calculate nearest-bench proximity. This allows the model to understand the distinct fuel profiles of high-resistance off-road zones versus maintained haul roads.

4. Leakage-Proof Temporal Dynamics: To capture the baseline health, degradation, and unique performance profile of individual dumpers, we integrated historical vehicle features. Crucially, this was executed using strict expanding().mean().shift(1) transformations. This mathematical constraint guarantees that the model only learns from historical data prior to the current shift, entirely eliminating look-ahead bias and future-data leakage during cross-validation.

5. Two-Stage Bimodal Architecture: Mining fuel data is inherently bimodal, heavily skewed by zero-fuel idle or maintenance shifts. Training a standard regressor on this data forces the model to artificially lower its predictions for working shifts. To solve this, we decoupled the problem:

6. SLSQP-Optimized Heterogeneous Ensemble: Rather than relying on naive equal-weight averaging for our final predictions, we implemented Sequential Least Squares Programming (SLSQP). This constrained mathematical optimization actively learns the optimal blending weights for our ensemble members (XGBoost, CatBoost, LightGBM, etc.) based on out-of-fold validation, dynamically penalizing weaker models and rewarding robust ones to minimize final RMSE.

- **Stage 1 (Classification):** A gating classifier isolates and identifies non-working/idle shifts.

- **Stage 2 (Regression):** A dedicated regressor accurately predicts fuel volume exclusively for active working shifts.

7. Strict Generalization & Overfit Prevention: To ensure stable performance on temporally distant, unseen data (the leaderboard), we implemented rigorous early-stopping protocols across all gradient boosting estimators. By combining a lowered learning rate with an explicit eval_set patience threshold, we prevented catastrophic overfitting, mitigating edge-case RMSE explosions and ensuring our models generalize safely to future fleet conditions.

Reproducibility

Component	Version / Notes
Python	3.10 or 3.11
OS	Windows 10/11 (update RAW path at top of script)
Hardware	Multi-core CPU strongly recommended (joblib geo parallelisation)
Approx. runtime	45–120 min depending on core count (dominated by geofencing + CatBoost)

Steps to reproduce

1. Install dependencies:

```
pip install pandas numpy pyarrow pyproj scikit-learn xgboost lightgbm catboost scipy joblib
```

2. Update the RAW path constant at the top of ai_guild.py to your local data directory.

3. Run:

```
python ai_guild.py
```

4. Five output files will be written to the data directory (primary + four secondary CSVs).

We have created a consolidation of all our previous codes and it includes what went wrong in them, what we tried, what failed and what we fixed.

https://docs.google.com/document/d/10qUVsKHuyPz6qT-pJIGrYLNfszMGXye_/edit?usp=sharing&ouid=102247205529967413724&rtpof=true&sd=true